

Computer Graphics DT082A Lab - 2 Report

Student: Kaan Tekin Öztekin

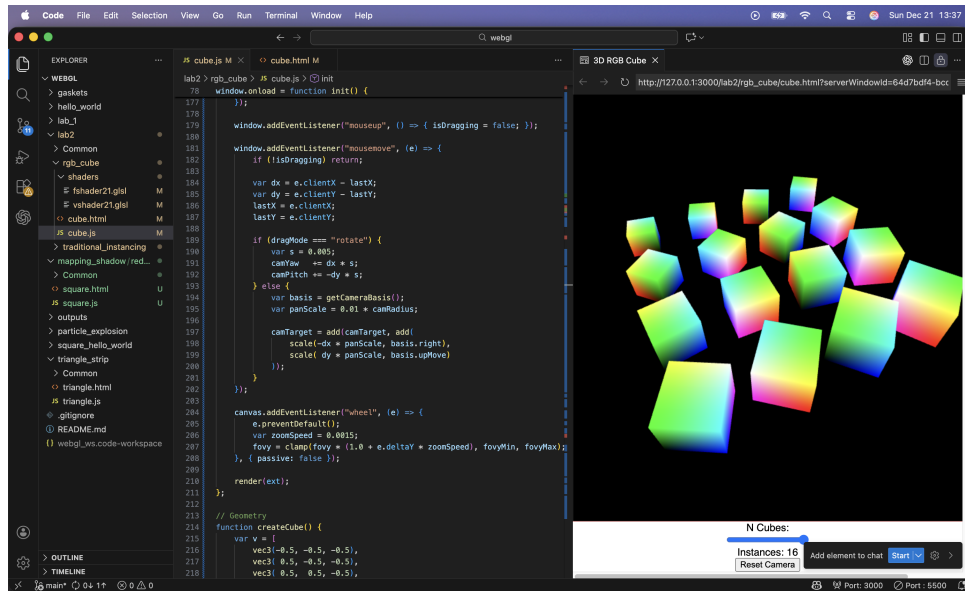
Date of submission: 21.12.2025

Implementation Process

This laboratory builds directly upon the single-cube implementation developed in Lab 1. While the initial exercise focused on understanding the basic WebGL pipeline, the second lab extends the scene to include multiple objects, interactivity, and a proper camera system.

The most significant architectural change was the transition from rendering a single cube using a traditional draw-call-based approach to rendering multiple cubes efficiently using instanced rendering. Instead of duplicating geometry data or issuing multiple draw calls, a single cube mesh is uploaded to the GPU once and rendered multiple times using per-instance transformation matrices.

Additionally, an interactive camera system inspired by a trackball model was implemented. Mouse input allows the user to rotate the camera around the scene, pan the view, and zoom by adjusting the field-of-view. A reset button restores the camera to its initial configuration.



Traditional vs Instanced Rendering

Initially, multiple cubes were considered using the traditional rendering approach, where each object is drawn separately by updating the model matrix uniform and issuing a new draw call. While this method is conceptually simple, it scales poorly as the number of objects increases, since each cube requires its own CPU–GPU communication and draw invocation. To address this limitation, instanced rendering was adopted using the `ANGLE_instanced_arrays` extension. In this approach, the cube geometry is sent to the GPU only once, while an array of model matrices is provided as per-instance attributes. The GPU then renders multiple instances of the same geometry in a single draw call.

Matrix Multiplication Order and Its Impact

One of the most important conceptual insights gained during this lab was the effect of matrix multiplication order on object transformations.

When the model matrix was constructed as:

$$M = R \cdot T$$

the cube rotated around the world origin rather than rotating around its own local center. This caused the object to orbit the scene instead of spinning in place.

Reversing the order to:

$$M = T \cdot R$$

resulted in the desired behavior, where each cube rotates around its own axis while remaining fixed at its assigned position in the grid.

Obstacles and Challenges

- Understanding how instanced rendering works in WebGL required careful handling of attribute buffers and stride offsets, particularly when passing a `mat4` as four separate `vec4` attributes.
- Incorrect stride and offset values initially resulted in `INVALID_VALUE` and `INVALID_OPERATION` WebGL errors, highlighting the importance of precise byte-level memory layout.
- Implementing a trackball-style camera introduced challenges related to camera flipping and numerical instability, which were resolved using clamping on pitch angles and field-of-view values.

Lessons Learned

- I gained a deeper understanding of modern rendering pipelines, particularly how instanced rendering reduces draw calls and improves performance.
- I learned that transformation matrices must be carefully ordered, as even small changes in multiplication order can completely alter object behavior.
- Implementing an interactive camera emphasized the importance of numerical stability and constraint handling in user-driven systems.